

CX-AICODE 功能讲解

企业级 AI 编程插件库与标准研发流程

围绕项目初始化、需求澄清、任务执行、代码审查、验证交付的全流程协同能力



核心流程

6



主命令

12+



内置 Agents

8



多技术栈支持





项目定位：把 AI 编程从“个人技巧”变成“企业流程”

核心价值是流程固化、质量门禁、审查留痕和可审计交付

流程治理

需求探索 / 规格定义 / 执行 / 审查 / 验证 / 收尾形成闭环，让工作可追踪、可复盘。

质量门禁

CodeReview、Verify、TDD、Debug 纪律把“能跑”升级为“合规可交付”。

企业审计

关键命令调用、SPEC-ID 绑定、Git 身份识别、本地与集中上报，形成治理依据。

知识复用

技术 Skill、业务 Skill、Reference、Codebase Skill 共用沉淀项目能力基线。

核心定位

CX-AICODE 不是单个代码生成命令，而是一套围绕 SPEC-ID、项目知识库、只读审查 Agent、Hook 门禁与交付记录构建的企业级 AI 编程作业系统。



当前项目资产地图

从命令、Skills、Agents、Hooks 到服务端审计的完整插件工程

项目文件

330

Skill 文件

91

命令

27

Agents

8

cx-aicode/
commands/ 27
agents/ 8
skills/
process | tech
quality | business

用户入口

27 个 /cx-* 命令，定义触发场景、用法与边界

流程能力

brainstorm / work / review / verify / test / doc / diagram

知识层

技术栈规范、质量规则、MES/EAP 业务技能

只读分析

Read / Grep / Glob，不写文件、不改状态

治理支撑

Hooks 注入上下文与门禁，Usage API 集中上报



01 主流程单页讲解

从项目初始化到交付收尾，逐页解释主链路命令与执行方式

流程闭环

命令能力

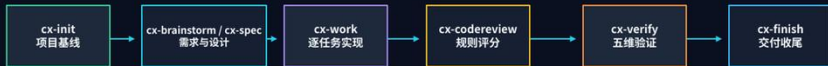
质量与验证

知识沉淀



核心流程总览

从需求澄清到验证交付的闭环研发流程



输入

项目目录 / 需求说明 / 规范 ID / 已批准设计 / 变更代码 / 新鲜测试证据

核心控制

HARD-GATE 设计批准、三文件、只读审查 Agent、Reference Inventory、原子状态更新

输出

.cx/specs、.cx/work、.cx/reports/codereview、.cx/reports/verify、交付记录

企业价值

将 AI 编程过程变成可讲解、可复盘、可审计、可持续改进的流程资产



主命令 1：cx-init 项目初始化

建立项目级 .cx 基线、复制技能模板、注册 Hooks，作为所有流程的起点

/cx-aicode:cx-init

定位 / 价值

新项目或旧项目首次接入 CX-AICODE 时执行。支持 --tech、--flavor、--minimal、--force 等模式。

核心产出

.cx/config.json、flavor.json、specs、docs、diagrams、status、archive、skills、templates。

门禁 / 注意

只允许一次 Bash 工具调用，避免模型逐个文件手工创建初始化产物。

功能要点

- 01 按技术栈或 flavor 复制项目级 skills 与模板
- 02 初始化只建立治理基线，不提前创建 .cx/work 三文件
- 03 插件级 hooks 自动注册，覆盖 SessionStart / PreToolUse / PostToolUse 等事件
- 04 适合大型组织统一分发标准流程和参考规则

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-init --tech=java vue  
/cx-aicode cx-init --flavor=mes  
/cx-aicode cx-init -m minimal
```



主命令 2：cx-brainstorm 需求探索

把需求说明变成 design.md + tasks.md，并强制设计批准后才能实现

/cx-aicode:cx-brainstorm

定位 / 价值

适合新需求、优化需求、用户想法尚需澄清的场景。围绕目的、约束、成功标准进行一次一问式探索。

核心产出

.cx/specs/{SPEC-ID}/source.md、
sources/、metadata.yaml、
design.md、tasks.md、approval-
state.json。

门禁 / 注意

HARD-GATE：设计文档未获得显式批准前，禁止写任何实现代码。

流程拆解

- 01 绑定外部需求 ID，导入 source 文档
- 02 一次一问澄清需求，探索 2-3 种方案
- 03 生成设计与任务清单，approval-state=pending_approval
- 04 只有“批准设计 / APPROVE”等显式语义才进入实现

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode:cx-brainstorm --spec-id MES-0535 \  
--source docs/需求.md --type 产品需求
```



主命令 3：cx-spec 规范定稿

面向正式规范、Delta Spec 与 OpenSpec 生命周期管理

`/cx-aicode:cx-spec`

定位 / 价值

当需求需要形式化管理、增量变更合并、跨团队协作或长期归档时使用。

核心产出

完整规范：proposal.md + design.md + tasks.md；Delta Spec：delta-spec.md。

门禁 / 注意

需显式 SPEC-ID；规范归档与合并由 openspec.ps1 / merge-delta-spec.ps1 支持。

两种模式

- 01 完整规范：适合新功能、跨团队、高风险变更
- 02 Delta Spec：适合已有系统增量修改
- 03 支持 ADDED / MODIFIED / REMOVED / RENAMED
- 04 PowerShell 原生 CLI，Windows 环境无需 Node.js 依赖

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-spec new --spec-id M ES-0535  
/cx-aicode cx-spec new --spec-id O PT-0123 --  
delta
```




主命令 4：cx-work 逐任务执行

实现子代理 + 规范合规审查 + 代码质量审查的任务循环

`/cx-aicode:cx-work`

定位 / 价值

在已批准设计和 tasks.md 存在后执行。每个任务必须按顺序实现、审查、修复、再审查。

核心产出

.cx/work/task_plan.md、findings.md、progress.md、cx-work-session.json，并持续记录变更文件。

门禁 / 注意

同一时间只运行一个实现子代理；规范合规审查通过后才能进入代码质量审查。

每任务循环

- 01 自动发现或指定 SPEC-ID，初始化三文件
- 02 实现子代理完成局部任务和自检
- 03 cx-work-spec-reviewer 只读审查“是否构建正确”
- 04 cx-work-quality-reviewer 只读审查“是否良好构建”
- 05 任务通过后强制更新三文件，再进入下一项

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-work --spec=M ES-0535  
/cx-aicode cx-work --task=1.1  
/cx-aicode cx-work --status
```



主命令 5：cx-codereview 代码审查门禁

基于 Reference Inventory、变更范围和评分规则生成结构化审查报告

/cx-aicode:cx-codereview

定位 / 价值

日常开发默认按 spec 或 changed 审查；
发布 / 基线可全量审查；历史追溯可用
marker 范围。

核心产出

.cx/reports/codereview/CodeReview-
*.md，更新 reviewStatus、
qualityGates.codeReview。

门禁 / 注意

调用 reviewer 前必须生成 Reference
Inventory；状态更新必须通过原子脚本且校
验 ok:true。

审查范围

- 01 spec：读取 design/tasks/ 三文件 /git diff，聚焦当前需求
- 02 changed：基于分支差异、时间或提交数审查
- 03 marker：基于源码 SPEC-ID 注释定位历史变更范围
- 04 full：适合发布前、质量基线或专项治理

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-codereview --scope=spec --  
spec=M ES-0535  
/cx-aicode cx-codereview --scope=marker --  
spec=M ES-0535
```



主命令 6：cx-verify 五维验证

没有新鲜验证证据，不能声称完成

`/cx-aicode:cx-verify`

定位 / 价值

在 CodeReview 通过后，对设计与实现进行需求覆盖、接口、数据、业务规则、异常处理五维验证。

核心产出

`.cx/reports/verify/verification-report-{SPEC-ID}-.md`，更新 `verifyStatus` 与 `reports.latestVerification`。

门禁 / 注意

缺少新鲜测试 / 构建证据时必须返回 `insufficient_evidence`，禁止写入 `passed`。

五维验证表

- 01 D1 需求覆盖：tasks.md 对照实现证据
- 02 D2 接口契约：设计 API 与 Controller/Service 对比
- 03 D3 数据模型：DTO/Entity 字段、类型、关系一致性
- 04 D4 业务规则：设计流程与实现逻辑一致性
- 05 D5 异常处理：已知风险与处理策略覆盖

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-verify --spec=M ES-0535  
/cx-aicode cx-verify --dm=apidata
```



主命令 7：cx-finish 交付收尾

将 AI 编程任务从“做完”转为“可交付、可复盘、可归档”

`/cx-aicode:cx-finish`

定位 / 价值

在实现完成、CodeReview 和 Verify 均满足后，执行本地交付收尾流程。

核心产出

交付记录、遗留问题清单或归档记录；版本控制仍交由企业既有研发流程。

门禁 / 注意

Step 0 强制检查 reviewStatus 与 verifyStatus；测试失败时停止。

三种收尾选项

- 01 标记完成：纳入企业交付记录
- 02 暂不完成：生成遗留问题清单，保留当前工作
- 03 废弃任务：仅归档过程记录，不纳入交付
- 04 废弃任务必须输入 discard 进行二次确认

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-finish  
# Step 0: Review + Verify Gate
```

02 重要补充命令

围绕测试、QA、调试、文档与知识沉淀的扩展能力

测试/QA

调试定位

文档图表

代码分析



补充命令：cx-test 自动化测试工件生成

从规范和代码生成测试计划、用例、数据、追踪矩阵和补强队列

`/cx-aicode:cx-test`

定位 / 价值

面向单元、集成、E2E 等自动化测试工件；
优先读取 spec/design/tasks 进行增强。

核心产出

.cx/qa/{module}/test-plan.md、scenario-
matrix.md、requirement-
traceability.md、test-cases.md、
summary-report.md。

门禁 / 注意

自动检测不足时必须显式 `--lang`；执行后可
基于反馈自动生成 regenerated 补强产物。

适配框架

- 01 Java：JUnit5 + Mockito / Spring Boot Test
- 02 Vue：Vitest + MSW / Playwright
- 03 C++：Google Test / 模拟器测试
- 04 VB.NET：NUnit / FlaUI
- 05 Python：pytest；C#：xUnit / WebApplicationFactory

核心价值

将流程、规则、状态、审查和交付产物
固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-test --lang=java --type=all  
RUN_TESTS=0 / PROMOTE_TESTS=1
```



补充命令：cx-qa 手工 QA 规划

生成测试计划、手工用例、回归套件、Figma 验证和缺陷报告

/cx-aicode:cx-qa

定位 / 价值

文档级手工 QA 规划，适合验收、回归、设计实现差异核对和缺陷记录。

核心产出

默认输出到会话；显式 --write 时写入 .cx/reports/qa/。

门禁 / 注意

不生成可执行测试代码，不影响 reviewStatus / verifyStatus。

与 cx-test 的分工

- 01 cx-test：代码级自动化测试工件
- 02 cx-qa：手工测试、验收清单、缺陷报告
- 03 Figma 设计验证可覆盖尺寸、颜色、间距、交互状态
- 04 适合接入 Jira / Linear / GitHub Issues / Confluence / TestRail

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-qa --plan --feature=登录  
/cx-aicode cx-qa --cases --feature=登录 --count=5
```



补充命令：cx-debug 根因调试

未完成根因调查，不能提出修复

`/cx-aicode:cx-debug`

定位 / 价值

遇到错误、异常、不符合预期行为时使用。
先调查、再假设、再最小化验证、最后修复。

核心产出

根因分析、证据链、失败测试、单一修复和验证结果。

门禁 / 注意

3 次修复失败后必须停下，质疑架构与共享状态，而不是继续盲改。

四阶段纪律

- 01 根本原因调查：完整读错误、复现、收集证据
- 02 模式分析：寻找有效类似代码，对比差异
- 03 假设与测试：一次只验证一个假设
- 04 实现修复：先建失败测试，再做单一修复，再验证

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-debug --  
error="NullPointerException..."  
# NO FIXES WITHOUT ROOT CAUSE
```




补充命令：cx-diagram 图表生成

默认输出标准 PlantUML，并强制写入 .cx/diagrams/

/cx-aicode:cx-diagram

定位 / 价值

生成架构图、时序图、类图、流程图、状态图等，支持模板驱动和代码分析辅助。

核心产出

.cx/diagrams/overview.puml、{module}-sequence.puml、{module}-class.puml 等。

门禁 / 注意

默认不得输出 Mermaid；必须使用 @startuml/@enduml；不得只在对话中贴代码块。

功能亮点

- 01 项目级模板优先于系统模板
- 02 PlantUML 基础语法便于飞书、VS Code、在线服务器渲染
- 03 支持 MES 专用 SECS 时序、设备状态、Lot 流程
- 04 图表内容必须基于代码结构或用户描述，避免泛化占位符

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-diagram --type=overview  
/cx-aicode cx-diagram --type=class --from -code
```



补充命令：cx-doc 文档生成

模板驱动生成需求、设计、用户手册、API、安装部署等文档

`/cx-aicode:cx-doc`

定位 / 价值

把代码结构、项目信息与模板结合，生成面向交付、运维和用户使用的 Markdown 文档。

核心产出

`.cx/docs/requirements.md`、
`design.md`、`manual.md`、`api.md`、
`installation.md`。

门禁 / 注意

项目级 `.cx/docs/templates` 优先；占位符自动替换日期、项目名、版本、作者、SPEC-ID。

典型用途

- 01 需求规格说明书与设计文档补齐
- 02 API 接口文档可从代码分析生成
- 03 安装部署文档、用户使用手册模板化
- 04 配合 `cx-doc-check` 做模板合规检查

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-doc --type=design  
/cx-aicode cx-doc --type=api --from <code>
```



补充命令：cx-analyze 代码分析

六技术栈结构扫描与 Python AST 增强，输出结构、依赖、模式和指标线索

/cx-aicode:cx-analyze

定位 / 价值

为文档、图表、测试生成和代码理解提供结构摘要；不是严格审计工具。

核心产出

结构概览、依赖线索、复杂度估算、模式 / 反模式线索；可生成 doc/diagram/test 草案。

门禁 / 注意

Java/Vue/C++/VB.NET/C# 多为轻量扫描，Python 为 AST；生成物需要人工复核。

分析维度

- 01 类、函数、接口、模块结构
- 02 import/include、继承、字段 / 方法引用等静态关系线索
- 03 内部 / 外部依赖、循环依赖提示
- 04 设计模式、架构模式、反模式启发式匹配
- 05 复杂度、长度、耦合度等启发式指标

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode:cx-analyze --output=m arkdow n  
/cx-aicode:cx-analyze --generate=all
```



补充命令：cx-codebase-skill 项目级源码导航

为旧项目建立“先定位、再读源码、再修改”的导航层

/cx-aicode:cx-codebase-skill

定位 / 价值

生成 skills/tech/codebase/，辅助定位模块、理解结构、查找常见修改路径。

核心产出

skills/tech/codebase/INDEX.md 及模块摘要、导航、bugfix-playbook 等辅助资料。

门禁 / 注意

Codebase Skill 只做辅助定位；实际修改前仍必须读取真实源码。

适用场景

- 01 数百万行旧项目需要快速找到修改入口
- 02 项目成员知识分散，需沉淀代码库导航
- 03 cx-work 和 cx-debug 可优先读取索引辅助定位
- 04 refresh/check 机制用于刷新与完整性检查

核心价值

将流程、规则、状态、审查和交付产物固定下来，降低 AI 编程过程的不确定性。

```
/cx-aicode cx-codebase-skill build  
/cx-aicode cx-codebase-skill refresh  
/cx-aicode cx-codebase-skill check
```

03 值得加入讲解的底层能力

用 Reference、Skills、Agents、Hooks 解释为什么流程能稳定运行

[Reference](#)[Skills](#)[Agents](#)[Hooks](#)



Agent-first 只读审查模式

分析、审查与验证由只读 Agent 判断；写报告和改状态由主流程负责

code analyzer

code reviewer

verifier

QA reviewer

doc compliance

plan reviewer

spec reviewer

quality reviewer

安全边界

所有 agent 工具仅为 Read / Grep / Glob：不写文件、不运行命令、不委托其他 agent。

职责分离

Agent 输出 cx-agent-result；主流程负责校验结果、写报告、用原子脚本更新 status。



Hooks：把流程纪律嵌入 Claude Code 运行时

6 类事件 × Bash / Windows 双实现，负责上下文注入、准入门禁和错误追踪

SessionStart

读取状态、恢复上下文

UserPromptSubmit

注入当前任务上下文，统计命令调用

PreToolUse

审批 / cx-work / 三文件准入门禁

PostToolUse

有效 cx-work 会话下更新
progress/findings

Stop

会话结束统计与收尾提示

ErrorOccurred

错误恢复与失败路径记录

讲解要点：成功 PostToolUse 不重写 .cx/status/index.json，减少读后编辑冲突；错误路径和有效 cx-work 会话才维护必要上下文。



Reference + Skills：项目级规则优先，技术栈与业务知识补位

形成“编码规范、评分标准、技术栈、业务域、代码库导航”的多层知识基线

reference/	项目级 coderule 与评分标准，CodeReview 最高优先级依据
skills/tech/*	C++ / Java / Vue / Python / C# / VB.NET 六技术栈编码规范
skills/quality/*	各技术栈质量规则、测试与可维护性约束
skills/business/*	MES / EAP 等领域知识、术语与业务规则
skills/tech/codebase/	由 cx-codebase-skill 生成的项目源码导航辅助层





企业审计：命令调用统计与集中上报

静默记录关键 /cx-* 命令，支撑流程遵循度分析和企业级报表

cx-spec

cx-brainstorm

cx-work

cx-codereview

cx-verify

cx-test

cx-debug

本地事件

command-events.jsonl
完整事件流水

待上传

pending-events.jsonl
等待上传

服务端确认

acked-events.jsonl
确认回执

失败重试

upload-backoff.json
退避状态

本地报表

reports/
JSON / Markdown / CSV

输出成果

- 流程遵循度报表
- 命令调用统计
- SPEC-ID / 用户维度分析
- 支撑企业级治理看板

隐私边界

不上报 prompt、源码内容、完整本地路径或原始需求文件；
projectId 使用路径 SHA-256 哈希。



技术栈支持：六栈规范 + 领域规则组合

支持新项目、旧项目与混合技术栈，通过 --tech / --flavor / --with 进行初始化组合

C++

C++11 / SiView / CORBA

Java

JDK 17 / Spring Boot / MyBatis-Plus

Vue

Vue 3.4+ / TypeScript / Vite / Pinia

Python

Python 3.9+ / FastAPI / Pandas

C#

.NET 6/8 / C# 10/11/12

VB.NET

.NET Framework / HSMS / SECS-I

典型命令

```
/cx-aicode:cx-init --tech=java,vue  
/cx-aicode:cx-init --flavor=mes
```

组合方式

- 技术栈规范与业务域技能可叠加
- Java/Vue 项目不需要 MES/EAP
- 旧项目可用 flavor 快速带入业务知识



其它可纳入展示的命令

这些命令不是主链路必需，但能提升企业落地可维护性

cx-doctor

进入诊断：检查 .cx、hooks、agents、reference、usage、Git 身份与 SPEC-ID

cx-status

查看当前进度、质量状态、维护状态和当前 SPEC-ID

cx-feedback

处理 CodeReview / Verify / 计划审查中的反馈与待关闭项

cx-tdd

RED-GREEN-REFACTOR 测试驱动开发纪律，cx-work 默认可启用

cx-learn

沉淀错误、规范、反模式、性能与调试经验

cx-update

管理员维护：检查、同步、回滚、离线版本元数据



展示顺序：先看“闭环”，再看“能力包”

先展示流程闭环，再展示命令能力与治理价值

1

1. 业务痛点

需求不清、AI 乱改、审查难复盘、旧项目难定位、交付不可审计

2

2. 主流程闭环

init → brainstorm/spec → work → codereview → verify → finish

3

3. 关键门禁

HARD-GATE、三文件、Reference Inventory、五维验证、新鲜证据

4

4. 辅助能力

test / qa / debug / diagram / doc / analyze / codebase-skill

5

5. 企业治理

只读 Agents、Hooks、usage 审计、技术栈与项目知识库维护

结论：CX-AICODE 把“AI 帮我写代码”升级为“AI 在企业流程内受控完成研发任务”。

讲解结论

适合作为企业级 AI 编程能力说明、内部宣讲与方案演示的统一材料

CX-AICODE 的核心不是单条命令，而是把需求、实现、审查、验证、交付和审计串联成一套可执行、可复盘、可治理的企业级 AI 研发体系。

流程闭环

从 init 到 finish

命令能力

主链路 + 补充命令

质量门禁

审查 / 验证 / 证据

企业治理

审计、Hooks、只读 Agent

知识沉淀

Reference / Skills / Codebase Skill

推荐收尾话术：先用一个 SPEC-ID，从 cx-brainstorm / cx-spec 启动，再用 cx-work 贯彻实现，并以 CodeReview 与 Verify 闭环。